

时序差分学习

先讲一个故事

你每天走同一条路去学校。走到一半发现路被封了，你绕路，结果比平时多花了 10 分钟。下次你会怎么预测今天要花多久？

- **方法 A**：走完整条路，到家看钟，用实际时间更新预测。准确但必须等走完。
- **方法 B**：走到一半，看一眼剩下一半的路况，猜今天大概要多久，当场修正预测。不用等走完，但你猜的那部分可能有偏差。

方法 A 是蒙特卡洛 (MC)，方法 B 就是**时序差分学习 (Temporal Difference Learning, TD)**。

TD 的核心想法：不用等到结局，每走一步就用「即时反馈 + 对未来的猜测」来修正当前估计。你不需要知道完整的真相，只需要一个步骤的反馈和当前的预测。

TD 预测——一步更新法则

回顾：两个极端

蒙特卡洛 (MC)：跑完整条轨迹，用真实回报更新。

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

其中 $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$ 是从 t 到终点的实际累积奖励。

优点：无偏估计 (G_t 是 $v_\pi(S_t)$ 的无偏估计)。缺点：方差大 (一条轨迹波动大)，必须等 episode 结束。

动态规划 (DP)：用已知模型对所有后继状态求期望。

$$V(s) \leftarrow \sum_a \pi(a | s) \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

优点：零方差，确定性。缺点：需要完整的环境模型 $P(s' | s, a)$ ，现实中通常没有。

TD(0)：取两者的优点

TD(0) 的核心更新法则：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- $R_{t+1} + \gamma V(S_{t+1})$ 叫 **TD 目标 (TD target)**：用一步真实奖励 + 对后继状态的当前估计来近似 G_t 。
- $R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ 叫 **TD 误差 (TD error)**：当前估计和更「新鲜」估计之间的差距。

和 MC 的区别：MC 用真实 G_t ，TD 用估计 $V(S_{t+1})$ 替代从 $t+1$ 到终点的未知奖励。这就是 Bootstrapping —— 用估计值更新估计值。

和 DP 的区别：DP 对 s' 求期望需要分布 $P(s' | s, a)$ ，TD 只采样了一次实际的 S_{t+1} 。DP 需要模型，TD 不需要。

TD 误差的严格定义

设时刻 t 智能体在 S_t ，执行动作后得到 R_{t+1} ，转移到 S_{t+1} 。TD 误差 δ_t 定义为：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

如果 $V = v_\pi$ （真实价值函数），则 $\mathbb{E}[\delta_t | S_t = s] = 0$ 。这是 Bellman 方程的推论：

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$$

当 V 近似 v_π 时， δ_t 趋近于零均值。TD 学习就是不断将 δ_t 驱向零的过程。

MC 与 TD 的理论对比

	MC	TD(0)
更新目标	G_t (真实回报)	$R_{t+1} + \gamma V(S_{t+1})$ (一步估计)
Bootstrapping	否	是
偏差	无偏	有偏 ($V(S_{t+1})$ 不准)
方差	高 (G_t 波动大)	低 (只依赖一步 R_{t+1})
需等 episode 结束	是	否 (每步即可更新)
收敛速度	慢	通常快

核心权衡：TD 引入一个有偏但低方差的估计替代品，换取无需等待 episode 结束的能力和更快的收敛。在绝大多数实际问题中，这个权衡是划算的。

为什么 TD 通常收敛更快？

考虑一个简单例子：8 个 episode 的回报序列，真实 $v_\pi = 0.5$ 。MC 的更新目标 G_t 在每个 episode 内波动极大，而 TD 目标 $R_{t+1} + \gamma V(S_{t+1})$ 只依赖单步转移，波动范围受限于即时奖励的幅值。

从优化角度看：MC 最小化均方训练误差（对所有见过的回报做最小二乘）；TD 利用 Markov 性质，拟合 Bellman 方程所暗含的一致性约束——它找到的是极大似然 MDP 模型的真实解。实践中这个一致性约束几乎总是带来更好的泛化。

TD 控制——从预测到决策

预测问题（估计 v_π ）使用 **TD(0)**。但 RL 的目标是找到最优策略，需要从「评估给定 π 」上升为「在评估的同时改进策略」。

这就是 TD 控制：使用 TD 方法学习动作价值函数 $Q(s, a)$ 。

核心困难：我们不能直接对价值函数取 $\max$ 来选动作（像 DP 那样），因为我们不知道 $P(s' | s, a)$ 。

解决方案：直接学习 $Q(s, a)$ ，用 TD 更新，然后配合探索策略选动作。根据**探索策略**和**学习策略**是否相同，分为两大流派。

on-policy: SARSA

核心思想：学什么就做什么，执行策略 = 学习目标策略。

SARSA 的名字来自每一步使用的五元组：**State** $\rightarrow$ **Action** $\rightarrow$ **Reward** $\rightarrow$ **State'** $\rightarrow$ **Action'**。

更新式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

动作选择通常用 ε -贪心：以 $1 - \varepsilon$ 选贪心动作，以 ε 随机均匀探索。

SARSA 的更新逻辑：

1. 在 S_t 根据 ε -贪心选 A_t ；
2. 执行 A_t ，得到 R_{t+1} ，到达 S_{t+1} ；
3. 在 S_{t+1} 根据 ε -贪心再选 A_{t+1} （这步**不执行**，只用它的 Q 值）；
4. 用 A_{t+1} 的 Q 值更新 $Q(S_t, A_t)$ ；
5. $S_t \leftarrow S_{t+1}$, $A_t \leftarrow A_{t+1}$ ，重复。

关键：SARSA 的目标 $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ 中的 A_{t+1} 来自**实际执行的策略（含探索）**。因此 SARSA 学习的是 ε -贪心策略下的 q_π ，考虑到了探索的代价。

off-policy: Q-Learning

核心思想：学习最优策略（贪心），但执行时用探索策略（ ε -贪心）。

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

和 SARSA 的唯一区别：目标中使用了 $\max_a Q(S_{t+1}, a)$ 而不是 $Q(S_{t+1}, A_{t+1})$ 。

- SARSA 的目标取决于 A_{t+1} （实际选的动作，含探索噪音）。
- Q-Learning 的目标总是假设下一步选最优动作，**无论实际探索做了什么**。

这就是 off-policy：学习的策略（贪心 = 最优）和执行的策略（ ε -贪心 = 探索）是**不同的**。

SARSA vs Q-Learning: 悬崖行走

经典的 Cliff Walking 环境揭示了两种算法的本质差异。

环境： 4×12 格子。起点在左下角，目标在右下角。底部一行中间 10 格是悬崖，踩中回到起点并罚 -100。普通步罚 -1。

最优路径：紧贴悬崖上方走最短路径，但一旦探索误入悬崖代价极大。

算法	学到的策略	原因
Q-Learning	贴悬崖走（最短路径）	学习最优策略，不考虑探索风险
SARSA	绕远路，远离悬崖	学到的是 ε -贪心策略，远离悬崖更安全

关键洞察：Q-Learning 假设下一步及以后所有步都是最优的，所以它认为踩悬崖不是问题（最优策略不会踩）。SARSA 知道实际执行时偶尔会探索，如果贴着悬崖走，一次随机探索就会导致大灾难。因此 SARSA 学到更**保守但安全**的策略。

这在实践中意味着什么？如果训练时 ε 大（探索多），SARSA 的保守性有优势。如果最终部署时完全贪心（ $\varepsilon = 0$ ），Q-Learning 学到的激进策略可能更优。选择取决于：你的 agent 在实际部署中还会不会犯错或遇到未见过的情况。

期望 SARSA (Expected SARSA)

期望 SARSA 是 SARSA 的一个改进版本。它在计算目标时不采样 A_{t+1} ，而是对所有可能动作求期望：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)]$$

优势：消除了 A_{t+1} 的采样方差。在 ε -贪心策略下：

$$\sum_a \pi(a | s) Q(s, a) = (1 - \varepsilon) \max_a Q(s, a) + \frac{\varepsilon}{|A|} \sum_a Q(s, a)$$

期望 SARSA 统一了 SARSA 和 Q-Learning：

- 当目标策略 π 是 ε -贪心 $\rightarrow$ 期望 SARSA (on-policy)。
- 当目标策略 π 是纯贪心 $\rightarrow$ 期望 SARSA 等价于 Q-Learning (off-policy)。

实验表明期望 SARSA 在绝大多数任务中优于传统 SARSA，计算开销的微小增加被方差下降带来的样本效率提升远远补偿。

Softmax 策略下的期望 SARSA

策略可用 softmax 而非 ε -贪心：

$$\pi(a | s) = \frac{\exp(Q(s, a)/\tau)}{\sum_b \exp(Q(s, b)/\tau)}$$

其中 $\tau > 0$ 为温度参数。 τ 大 $\rightarrow$ 趋于均匀探索； $\tau \rightarrow 0 \rightarrow$ 趋于纯贪心。

Softmax 策略的优势：探索不再是盲目的 (ε -贪心对 $\varepsilon/|A|$ 概率选任何动作，包括明显差的)，而是按 Q 值的相对大小分配探索概率——更好的动作获得更多探索。

N 步 TD ——在一步和全程之间

为什么需要 N 步？

TD(0) 只看一步：低方差高偏差。MC 看全程：无偏高方差。N 步 TD 是两者之间的连续光谱。定义 n 步回报：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- $n = 1$: $G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$ ——TD(0)。
- $n \rightarrow \infty$: $G_t^{(\infty)} = G_t$ ——MC。
- $1 < n < \infty$: n 个真实奖励 + 在第 n 步截断处用 V 估计余下部分。

N 步 TD 更新式：

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t^{(n)} - V(S_t)]$$

N 步 SARSA

同理，将 V 替换为 Q ， n 步回报定义：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$$

更新式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t^{(n)} - Q(S_t, A_t)]$$

期望版本 (Expected n -step SARSA): 最后一步用 $\sum_a \pi(a | S_{t+n}) Q(S_{t+n}, a)$ 代替 $Q(S_{t+n}, A_{t+n})$ ：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \bar{V}(S_{t+n})$$

其中 $\bar{V}(s) = \sum_a \pi(a | s) Q(s, a)$ 。消除了最后一个动作的采样方差。

TD(λ) —— 用衰减因子混合所有步长

与其只用一个 n ，TD(λ) 将所有 n 步回报用几何加权平均：

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- $\lambda = 0$: $G_t^\lambda = G_t^{(1)}$ —— TD(0)。
- $\lambda = 1$: $G_t^\lambda = G_t$ —— MC。
- $0 < \lambda < 1$: 越近的步权重越大，越远的步贡献越小但非零。

λ 的作用：控制了 bootstrap 的程度。 λ 越小，越依赖估计值（高偏差低方差）； λ 越大，越依赖真实奖励（低偏差高方差）。

资格迹——如何高效实现 TD(λ)

核心问题

TD(λ) 需要等到 episode 结束才能计算 G_t^λ （需要所有 n 步回报）。资格迹 (Eligibility Traces) 提供了一种在线、增量式的实现方式，每步即可更新而不需要回头看。

资格迹的直觉

每访问一个状态-动作对 (s, a) ，在它的「资格迹」上加 1。此后每步，所有资格迹都衰减 $\gamma\lambda$ 。TD 误差发生时不只更新当前的 (s, a) ，而是按资格迹的比例更新所有被访问过的对。

$$e_t(s, a) = \begin{cases} \gamma\lambda e_{t-1}(s, a) + 1, & \text{如果 } (s, a) \text{ 是当前对,} \\ \gamma\lambda e_{t-1}(s, a), & \text{否则.} \end{cases}$$

更新式：每一步对所有 (s, a) 执行：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t e(s, a)$$

其中 δ_t 是当前步的 TD 误差。

直觉：资格迹是「这个状态-动作对在多大程度上应该为当前的结果负责」的衰减记录。越近发生的对权重越大；久远的对影响逐渐衰减归零。一集开始前所有资格迹重置为 0。

Watkins' $Q(\lambda)$: off-policy 资格迹

资格迹天然就是 on-policy 的——记录的是实际执行的动作序列。Q-Learning 是 off-policy 的，需要特殊处理。

Watkins 的关键洞察：资格迹只能延续到当前执行的动作链与最优策略一致的时刻。一旦执行了一个非贪心动作（探索），未来轨迹不再遵循目标策略，资格迹必须被清零。

$$\text{若 } A_{t+1} \neq \arg \max_a Q(S_{t+1}, a) \implies \text{所有 } e(s, a) \leftarrow 0$$

后果：探索步截断资格迹，只有连续的贪心段被用于传播误差。这在确保理论正确性的同时，导致资格迹的有效长度取决于「连续贪心动作的长度」而非固定的 λ 参数——探索率高时效率降低。

经验回放——打破数据的时间相关性

问题

Q-Learning 是 off-policy 的——它可以从任何策略产生的数据中学习。但在线学习中，每一步只使用当前转移 $(S_t, A_t, R_{t+1}, S_{t+1})$ 更新一次然后丢弃。数据利用率极低。

经验回放缓冲

维护一个固定大小的**回放缓冲** (replay buffer)，存储最近的 N 条转移 $(s, a, r, s', \text{done})$ 。
每步：

1. 执行策略产生新转移，存入缓冲；
2. 从缓冲中随机采样一个 mini-batch；
3. 在采样的转移上做多次 Q-Learning 更新。

为什么有效？

- **打破相关性：**连续步的转移高度相关，随机采样打乱顺序，降低更新方差；
- **样本复用：**每条转移被多次使用，提高数据效率；
- **稳定训练：**相当于用了一组「旧数据」做正则化。

限制：经验回放只能用于 **off-policy** 算法 (Q-Learning、Expected SARSA)。SARSA 不能用，因为它的更新依赖当前策略下的动作选择，旧数据来自旧策略，不匹配。

TD 学习算法族谱

算法	on/off-policy	Bootstrap	目标形式	特点
MC	on	否	G_t	无偏, 高方差
TD(0)	on	是	$R + \gamma V(s')$	有偏, 低方差
SARSA	on	是	$R + \gamma Q(s', a')$	考虑探索代价
Q-Learning	off	是	$R + \gamma \max Q(s', \cdot)$	学最优策略
Expected SARSA	均可	是	$R + \gamma \sum \pi(a s') Q(s', a)$	低方差
N-step	均可	是	n 步混合回报	偏差-方差可调
$Q(\lambda)$	off	是	λ -加权回报 + 资格迹	在线多步

一句话总结每个算法

- **TD(0)**: 用「一步真实奖励 + 下一步估计」更新, 无需等 episode 结束。
- **SARSA**: 做什么就学什么, ϵ -贪心下的真实 q_π , 保守安全。
- **Q-Learning**: 直接学最优策略的 q_* , 不管实际如何探索。
- **Expected SARSA**: 对采样求平均, 降低方差, 比 SARSA 稳定且可退化为 Q-Learning。
- **N-step TD**: 折中偏差和方差, 越大的 n 越接近 MC。
- **TD(λ) + 资格迹**: 用指数衰减对每步误差分配功劳, 在线实现多步传播。
- **经验回放**: 存旧数据、随机重放, 打破相关性、提升样本效率 (仅 off-policy 可用)。

从 TD 到 DQN 的桥梁

理解本章为下一阶段做好准备:

1. Q-Learning 的 max 算子 + 函数逼近 = **DQN** (Deep Q-Network)。
2. 经验回放 + 目标网络 = 解决 DQN 中 TD 更新的不稳定性。
3. ϵ -贪心探索被延续到 DQN 中 (简单但有效, 后来被 NoisyNet 等方法改进)。
4. 资格迹的思想 $\rightarrow$ Retrace 算法 $\rightarrow$ 结合经验回放实现高效 off-policy 多步学习。

参考资料

1. Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapter 6 (TD Learning), Chapter 7 (n-step), Chapter 12 (Eligibility Traces).
2. Watkins, C. J. C. H. (1989). Learning from Delayed Rewards. PhD Thesis. (Q-Learning 的原始论文)
3. Rummery & Niranjan (1994). On-line Q-learning using connectionist systems. (SARSA 的原始论文)

4. van Seijen et al. (2009). A theoretical and empirical analysis of Expected Sarsa.
5. Lin (1992). Self-improving reactive agents based on reinforcement learning. (Experience Replay 的发明)
6. Mnih et al. (2015). Human-level control through deep reinforcement learning. *Nature*. (DQN + Experience Replay)
7. Practical RL, Yandex Data School. Week 3 —Model-Free Reinforcement Learning.